MASTER'S DEGREE PROJECT

Machine Learning for the refinement of orbital propagation

Authored by
José Calderón Valdivia

For the completion of
Máster en Ingeniería del Software: Cloud, Datos y Gestión TI

Supervised by
Daniel Ayala Hernández
Rafael Ayala Hernández

June call, academic year 2023/24

Acknowledgements

I want to thank my family for supporting me at all times and for always being there when I needed; without them, it would not have been possible.

To my friends, whose company has been a fundamental pillar along this year.

To my teachers for everything they have taught me.

To the entire DEAL research group for their constant support, help, and valuable feedback.

And I especially want to thank my mentors, Daniel Ayala and Rafael Ayala, for their help throughout the entire project.

Resumen

La predicción precisa de las posiciones de los satélites es fundamental para una amplia gama de aplicaciones, incluyendo comunicaciones, navegación, teledetección y planificación de misiones espaciales. Esta predicción proporciona información vital para el funcionamiento seguro y eficiente de estos sistemas. Sin embargo, predecir las trayectorias de los satélites presenta desafíos significativos debido a diversas fuerzas y perturbaciones que actúan en órbita.

Actualmente, existen dos tipos principales de modelos para la predicción de posiciones satelitales: los modelos de perturbaciones simplificadas (como SGP4 y SDP4, a los que nos referiremos conjuntamente como SGDP4) y los modelos de alta precisión (como HPOP). Aunque los modelos de alta precisión son más fiables, su elevado coste computacional los hace poco prácticos para predicciones a medio o largo plazo.

En la literatura existente, se han aplicado redes neuronales al campo de la predicción de trayectorias de satélites, pero estos trabajos se centran en desarrollar modelos que únicamente utilizan datos del propio satélite, sin considerar factores externos como las posiciones de los planetas o el Sol.

El objetivo de este proyecto es estudiar la viabilidad de utilizar redes neuronales para mejorar la precisión del modelo SGDP4. Para ello, se analizarán diversas combinaciones posibles de datos de entrada, incluyendo la consideración o no de la velocidad del satélite, y las posiciones de los astros. Además, se explorarán diferentes arquitecturas de redes neuronales, como redes densas, redes GRU y redes residuales. También se evaluarán distintas configuraciones del modelo, como predecir coordenadas por separado o entrenar para tiempos de predicción concretos.

Los resultados demostraron que la incorporación de técnicas de aprendizaje automático en la propagación orbital constituye una herramienta poderosa para mejorar la precisión y eficiencia de las predicciones orbitales. Aunque existen desafíos y limitaciones, los beneficios potenciales son considerables, indicando un camino prometedor para futuras investigaciones y aplicaciones en el campo de la dinámica orbital.

Palabras clave: Propagación de órbitas, Machine Learning, Redes Neuronales, Satélites, SGP4, SDP4

Abstract

Accurate prediction of satellite positions is crucial for a wide range of applications, including communications, navigation, remote sensing, and space mission planning. This prediction provides vital information for the safe and efficient operation of these systems. However, predicting satellite trajectories presents significant challenges due to the diverse forces and perturbations acting in orbit.

Currently, there are two main types of models for satellite position prediction: simplified perturbation models (such as SGP4 and SDP4, which we will collectively refer to as SGDP4) and high-precision models (such as HPOP). While high-precision models are more reliable, their high computational cost makes them impractical for medium or long-term predictions.

In the existing literature, neural networks have been applied to the field of satellite trajectory prediction. However, these works focus on developing models that only use data from the satellite itself, without considering external factors such as the positions of the planets or the Sun.

The goal of this project is to study the feasibility of using neural networks to improve the accuracy of the SGDP4 model. To achieve this, different possible combinations of input data are analyzed, including whether or not to consider the satellite's velocity and the positions of the celestial bodies. In addition, different neural network architectures are explored, such as dense networks, GRU networks, and residual networks. Different model configurations are also evaluated, such as predicting coordinates separately or training for specific prediction times.

The results demonstrated that the incorporation of machine learning techniques into orbital propagation constitutes a powerful tool for improving the accuracy and efficiency of orbital predictions. While there are challenges and limitations, the potential benefits are remarkable, indicating a promising path for future research and applications in the field of orbital dynamics.

Keywords: Orbit propagation, Machine Learning, Neural Networks, Satellites, SGP4, SDP4

Contents

List of Figures

1. Introduction

1.1. Introduction

In recent decades, the number of satellites orbiting Earth has grown remarkably, with over 11,700 active satellites currently in operation[1]. This has enhanced our ability to observe and communicate globally, but it has also considerably increased the amount of space debris. Managing these orbits has become a significant technical challenge due to the dynamic complexity of the space environment [3, 17, 23].

Orbit propagation involves calculating the position and velocity of a satellite at a given time from initial conditions [8]. The satellite's trajectory is influenced by various forces, mainly gravitational and centrifugal, as well as other perturbations such as the influence of the Sun, the Moon, and other celestial bodies [26]. To make these predictions, two main types of models are used: analytical models and high-precision models.

Analytical models, also known as simplified perturbation models [12], include models such as SGP4 (Simplified General Perturbations) and SDP4 (Simplified Deep-space Perturbations), which we collectively refer to as SGDP4. These models offer an approximate way to model the satellite's orbit, allowing for fast but less precise long-term predicitons. They are useful for applications where speed is crucial and accuracy can be sacrificed in favor of computational efficiency.

On the other hand, high-precision models, whose foundations can be found in [4, 20, 27], (such as HPOP, High Precision Orbit Propagator) use differential equations to model all the forces acting on the satellite. These models are extremely precise, but they require high computing power, making them more expensive to apply, especially for long-term predictions. These models consider in detail gravitational perturbations, solar radiation pressure, atmospheric drag, among other factors, providing a more accurate prediction of the satellite's orbit.

In this context, Machine Learning emerges as a powerful tool to address these challenges. Advances in Machine Learning algorithms and the availability of large volumes of data have the potential for the development of models that significantly improve the accuracy and efficiency of orbit propagation.

Among the various Machine Learning techniques, neural networks stand out for their ability to model complex and non-linear relationships in data, making them particularly suitable for prediction and classification tasks. Neural networks have been shown to outperform other Machine Learning algorithms in terms of accuracy and efficiency in these contexts [1, 2, 14, 16, 25]. For this reason, this work focuses on exploring how neural networks can be applied in conjunction with simplified perturbation models, specifically SGDP4, to optimize orbital prediction, reducing errors in predicted trajectories. Throughout this work, we will present various neural network

[1] https://www.unoosa.org/oosa/osoindex/search-ng.jspx

architectures designed to address this specific task, evaluating the performance of each one compared to the SGDP4 model and analyzing its impact on accuracy.

1.2. Context of this TFM

In this section, the following points are discussed:

— **Degree of difficulty or complexity of the topic**

Precise propagation of satellite orbits represents an ongoing challenge in the field of astrodynamics. Despite the existence of well-established traditional methods, these face limitations in terms of accuracy or efficiency. Our approach aims to overcome these barriers, proposing solutions that find a balance between accuracy and efficiency.

Classical orbital propagation models that are highly accurate are also computationally intensive, requiring significant processing power and time for medium and long-term predictions. In this context, the application of neural networks emerges as a promising solution, as they have the potential to reduce these requirements. Although the initial training of neural networks may demand large volumes of data, once trained, they can provide quick and computationally efficient results. However, their application in this field is relatively novel, as most current solutions are based on physical models.

— **Degree of achievement of the stated objectives**

All the defined goals and sub-goals have been successfully achieved. The completion of goal G1.1 regarding the state of the art is detailed in Section 3. Goal G1.2, which refers to the collecting of the dataset, is covered in Section 5.3. Goals G1.3, G1.4, G1.5, G1.6, G1.7 and G1.8, which involves the design of architecture, experiments and analysis of the results, can be found in Chapters 4 and 5.

— **Research component of the work**

After an exhaustive review of the state of the art, we have identified several gaps detailed in Section 3.4. This project aims to address these unexplored areas, generate valuable knowledge, and lay the foundation for future research. To achieve this, we have designed and evaluated various neural network architectures, testing their performance with different input parameters, including information from the satellite itself, data from the planets, or training them for specific time bands.

— **Potential application of this work in an academic or industrial enviroment**

This work has produced significant results and opened new lines of research, such as the search for more general solutions applicable to multiple satellites. In the industrial field, the developed network architectures could eliminate the need to use costly models for some tasks, generating considerable economic savings and allowing predictions to be made much faster.

1.3. Document structure

The rest of this document is structured as follows: Chapter 2 is dedicated to project management aspects, Chapter 3 details the state of the art in the fields of satellite propagation, focused on Machine Learning approaches. Chapter 4 presents our proposal, explaining in detail the architectures that have been developed. In Chapter 5, the configurations used in the experimentation are detailed, and the obtained results are presented and analyzed. Chapter 6 summarizes our contributions and explores potential future work.

2. Preliminar study

2.1. Introduction

This chapter focuses on the management aspects of the project, covering key elements that were addressed before execution. It includes a study of the technologies used, the objectives set, the methodology followed, and the initial estimates for time and cost planning. Additionally, it compares these initial estimates with the actual values obtained upon project completion.

2.2. Aims and scopes

The aim of this study is to assess whether integrating SGDP4 propagator with neural network models can enhance the accuracy of specific predictions.

This research will specifically examine the SGP4/SDP4 model, widely used for satellite orbit prediction. By combining traditional methods with neural networks, we aim to determine if such hybrid approaches can yield more precise results, particularly in contexts where satellite-specific data may be sparse or unavailable. This ensures that the neural network does not need to rely on potentially unavailable satellite-specific data, such as the area of the section, thereby enhancing its practical applicability.

This study involves a thorough analysis of the SGP4/SDP4 model's performance when augmented with neural network techniques, providing insights into potential improvements in predictive accuracy.

The proof of concept implemented in this study focuses on predicting the position of a specific satellite. By concentrating on a single satellite, this approach eliminates the necessity of providing the model with certain satellite-specific data that may not be readily accessible, such as the cross-sectional area, radiation area, or radiation pressure. This allows the model to implicitly learn and account for these specific values through its training process, despite not having direct access to this detailed information.

2.3. Goals

The main objective of this project is:

- **Goal G1: Study the influence of different architectures with different input data on neural network performance for satellite propagation.**

To do so, we have defined a set of sub-goals:

- **Goal G1.1**: Study the state of art by understanding neural networks architectures, variations and most popular approaches.
- **Goal G1.2**: Select or create a dataset with proper information to train neural network models.

- **Goal G1.3**: Determine the optimal combination of satellite characterictics for prediction.
- **Goal G1.4**: Evaluate the incorporation of celestial body data.
- **Goal G1.5**: Assess the use of sequential data.
- **Goal G1.6**: Employ models with separate coordinates.
- **Goal G1.7**: Implement models for different time frames.
- **Goal G1.8**: Analyze experiment results using visual elements such as charts to extract conclusions.

2.4. Methodology

An adaptation of classical software development methodologies, structured in different sequential phases, has been employed to meet the specific needs of the research and the particularities of a Master's thesis. This methodological approach has been designed to align with the defined objectives of the project, which in turn reflect its various phases. These phases bear a close resemblance to the stages of the research process and the typical structure of a scientific article in the field; beginning with a study of the current literature, the implementation of an architecture for relevant experimentation, the selection of suitable datasets, experimentation based on defined research questions, the analysis of the obtained results, and their dissemination.

Weekly monitoring of the project has been conducted through regular meetings with the tutors. During these meetings, progress is reviewed and results are evaluated against the planned objectives. Difficulties are also discussed, and effective solutions are proposed to overcome them. This collaborative approach allows for early identification of potential problems and facilitates the timely implementation of corrective measures. Additionally, these sessions provide a platform for receiving constructive feedback, which is crucial for the continuous improvement of the project.

2.5. Planning

The stipulated time for this Master's thesis is three hundred hours. In this project, this time was divided into different phases, based on the objectives defined in Section 2.2. The initial planning is shown in the following picture.

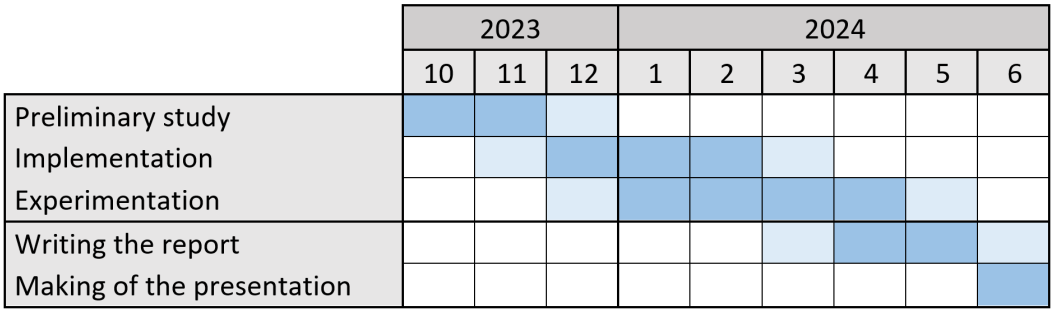

Figure 2.1: Schedule

Phase	Estimated dedication (hours)	Real dedication (hours)	Deviation	
			hours	%
Preliminary study	45	40	-5	-11.11
Implementation	115	125	10	8.69
Experimentation	100	113	13	13
Writing the report	30	27	-3	-10
Making of the presentation	10	12	2	20
Total	**300**	**317**	**17**	**5.67**

In addition to the traditional phases of research, the development of the project report and the presentation are included.

As observed, the initial estimates align closely with the actual hours invested. The most significant deviations occurred during the implementation and experimentation phases. These deviations are primarily due to some problems with the packages and the uncertainty of neural network training.

In the end, the initially planned 300 hours turned into a total of 317 hours.

2.6. Budget

The cost of this project includes several elements, the amount of which will depend on the time dedicated to the project.

Firstly, the personnel cost. A full-time computer engineer has an annual cost of approximately 31,000 euros[1] for 1800 hours per year, which equals 17.22 euros per hour worked.

On the other hand, within the equipment and supplies costs, the depreciation of the equipment used must be included. This consists of a laptop with an initial acquisition cost of 1200 euros, and a high-performance server with an i9 9900k, 64 GB of RAM, and an RTX 3080 graphics card, with an initial cost of 3200 euros. Both computers have an estimated useful life of four years, which is 48 months, and they will

[1]Value on 14 June 2024 on https://www.us.es/bous-numeros/numero-5-24-de-mayo-de-2023/acuerdo-61cg-13-4-23-normativa-contratacion-personal

be used for the project for eight months. Additionally, there are the electricity costs for the computers, with a combined estimated consumption of around 1000W/hour and an average cost per kilowatt-hour of 20 euro cents, and the internet connection costs, with a monthly rate of 30 euros.

In addition to the planned expenses, a risk reserve cost of 10 percent of the total is established to cover possible incidents during the execution of the project.

In this way, the estimated costs amount to a total of 6569.01 euros:

Item	**Calculation**	**Total**
Personnel cost	17.22 €/h x 300 hours	5166€
Depreciation of the equipment	4400 € / 48 months x 8 months	733.33 €
Electricity supply	1kWh x 0,20 € x 300 hours	60 €
Internet connection supply	30 € / 720 hours/month x 300 hours	12.5 €
	Subtotal sum	**5971.83 €**
	Reserve costs (10%)	**597.18 €**
	Total	**6569.01 €**

And the final costs were:

Item	**Calculation**	**Total**
Personnel cost	17.22 €/h x 317 hours	5458.74 €
Depreciation of the equipment	4400 € / 48 months x 8 months	733.33 €
Electricity supply	1kWh x 0,20 € x 317 hours	63.4 €
Internet connection supply	30 € / 720 hours/month x 317 hours	**13.21 €**
	Total spending	**6268.68 €**
	Total planned with reserve	**6569.01 €**
	Surplus	**300.33 €**

It is concluded, therefore, that there has been a cost deviation of 296.85 euros compared to the planned expenses without the reserve, representing a 4.97% deviation between these costs. Thanks to the reserves, these expenses have been covered and there is finally a surplus of 300.33 euros, representing 4.57% of the total budget.

2.7. Conclusions

In this section, different aspects of project management have been introduced and analyzed.

3. State of the art

3.1. Introduction

In this section, we delve into the body of existing research and analyses related work, providing a comprehensive background that enriches our understanding of the domain and the problem at hand.

3.2. Propagation models

Traditionally, the prediction of satellite orbits has been based on models grounded in physics. These models leverage data from various sources, including the satellites themselves, atmospheric conditions, planetary movements, and other celestial bodies, to model and predict the trajectory of satellites as they orbit Earth. This approach to orbit prediction can be categorized into two primary model families: Simplified Perturbation Models and High Precision Propagators, also referred to as General Perturbation Models.

Simplified Perturbation Models are designed to provide a more generalized and less computationally intensive way of predicting satellite orbits. These models take into account the major forces acting on a satellite, such as gravitational forces from the Earth and the moon, solar radiation pressure, and atmospheric drag, but they do so in a simplified manner. This makes them particularly useful for applications where a high degree of precision is not critical, or when computational resources are limited. The simplifications in these models, however, mean that they may not accurately account for the subtle variations in the forces acting on satellites, leading to less precise orbit predictions over time.

Within this family of models, there are subdivisions tailored to the orbit's proximity to Earth. We can difference those whose are for near-Earth objects (orbital period of less than 225 minutes, or in other words, altitude of less than 5,877.5 km, assuming a circular orbit.) like SGP, SGP4 and SGP8; and those for far-Earth objects (from deep space) like SDP4 and SDP8. They are useful for applications where less precision is required or for preliminary analysis, particulary in short term predictions.

The SGP theory was largely based on Kozai [15], while SGP4 was primarily based on Brouwer [5]. These two theories are quite different, but both are still in use today. Neither originally included drag effects, so different treatments for atmospheric drag are used. The equations for these models are very extensive; therefore, they are omitted in this document. For more detailed information on the models, one can refer to Hujsak [13], and if one wants to see an implementation of them, it can be found in Hoods and Roehrich [12].

On the other hand, High Precision Propagators, or General Perturbation Models, are designed for applications where high accuracy in orbit prediction is essential. These models incorporate a more detailed and comprehensive analysis of the forces affecting

a satellite, including the gravitational influences of other celestial bodies, the effects of Earth's oblateness, and the variations in atmospheric density at different altitudes. By taking these and other factors into account in a more precise manner, these models are able to offer significantly more accurate predictions of satellite positions and trajectories. However, this increased accuracy comes at the cost of greater computational complexity and resource requirements.

One of this models is HPOP (High Precission Orbital Propagator) [3]. In that model, relativistic effects are taken into account to apply corrections and the total acceleration $\mathbf{a}$ of a satellite is expressed as the sum of various force components and effects:

$$\mathbf{a} = \mathbf{a}_{\text{Earth}} + \mathbf{a}_{\text{tides}} + \mathbf{a}_{\text{Moon}} + \mathbf{a}_{\text{Sun}} + \sum_{i=\text{planets}} \mathbf{a}_i + \mathbf{a}_{\text{SRP}} + \mathbf{a}_{\text{drag}} \tag{3.1}$$

where:

$\mathbf{a}_{\text{Earth}}$: Acceleration due to Earth's gravitational attraction
$\mathbf{a}_{\text{tides}}$: Acceleration due to Earth's ocean and solid tides
$\mathbf{a}_{\text{Moon}}$: Acceleration due to the Moon's gravitational attraction
$\mathbf{a}_{\text{Sun}}$: Acceleration due to the Sun's gravitational attraction
$\mathbf{a}_i$: Acceleration due to the gravitational attraction of planet i
$\mathbf{a}_{\text{SRP}}$: Acceleration due to solar radiation pressure
$\mathbf{a}_{\text{drag}}$: Acceleration due to atmospheric drag

Note: In planets we considered the ones in the solar system, except Earth, which due to its importance is considered apart

To sum up, the choice between Simplified Perturbation Models and High Precision Propagators depends on the specific requirements of the application, including the needed level of orbit prediction accuracy and the available computational resources. While Simplified Perturbation Models offer a more resource-efficient option with reasonable accuracy, High Precision Propagators are the ideal choice for tasks that demand the highest levels of precision in orbit prediction.

3.3. Neural network approachs

Unlike physics-based modelling, machine learning methods do not need to simulate environmental conditions and all factors influencing a satellite to predict its position in an explicit way. These models learn from large amounts of observed data, similar to human cognition in learning from past experiences to predict future events [9]. These models may not be enough precise as high precision propagators, but can be combined with other models to improve their performance.

Ren et al. [24] proposed a combination of LSTM neural network and linear regression. They use LSTM neural network in order to predict the orbital inclination, orbital eccentricity and average displacement, trainning one model to predict each variable. Besides, linear regression is used to estimate the ascending node, perigee angular

distance and near point angle. After predicting this six elements of the TLE orbit, the position of the spacecraft can be obtained by a simple transformation to know the error of this prediction. The inputs for the six models are the same, the six orbital elements, the timestamp where these elements correspond and the time interval within the prediction must be done. To train these models, data from the IRIDIUM 118 satellite was used. The data was divided into training and testing sets, with 80% for training and 20% for testing. In this LSTM network, the six orbital parameters were provided as a five-element sequence, while the test employed a twenty-element sequence.. The authors consider the predicted position error to be within an acceptable range and its variation to stay relatively stable against the progress of time.

Peng and Bai [21] explored the viability of employing a Support Vector Machine (SVM) regression model to forecast the error in a assumed dynamical model of the ENVISAT satellite. Initially, they modeled three stations to generate discrete measurements, when the target satellite is visible to them, acording to a "truth" dynamical model. Then they applied least squares estimation to get the state of the satellite. After obtaining estimations for all the tracks, the prediction process is straightforward, the SVM model was tasked with predicting the error in this forecast to refine the model's estimates. The inputs for the SVM model are the duration of the prediction; position and velocity at the current epoch; estimated drag coefficient at the current epoch; maximal measured elevation in the current track and the corresponding range and azimuth; predicted position and velocity. The SVM's output was a six-component vector detailing the error in position and velocity across each axis. The conclusion drawn from their study was that the SVM model, once trained, had limited applicability for predictions extending too far into the future. Therefore, they advised that orbit predictions should be made within a relatively short timeframe to ensure accuracy.

Peng and Bai [22] later introduced a neural network approach, employing dense neural networks that used identical inputs from the same satellite as described for the Support Vector Machine (SVM) model in the previous paragraph. The outputs of the neural network mirrored those of the SVM, with a notable modification: the approach involved training a different model for each component, resulting in a total of six unique models. They studied also the models' ability to generalize to future epochs and to different, but nearby, Resident Space Objects (RSOs) in subsequent epochs. In their analysis of this second task, the authors analysed the range of learning variables, excluding any variables whose ranges exhibited significant disparities between the training and testing datasets. Their findings suggested that the neural network models demonstrate strong generalization capabilities to future epochs. Furthermore, it was concluded that these neural networks could be generalized to a relatively broad spectrum of nearby RSOs not included in the training dataset, showcasing their versatility and potential for predictive accuracy in dynamic space environments.

Liu et al. [18] developed a hybrid model, combining SGP4 with autoencoders and random forest to reduce the propagating error. First, they used the encoder to obtain a representation of the distance error from SGP4. Then, a random forest model was applied to predict the embedding vector for the next time step and finally, a decoder obtained the distance error. This was done for each positional coordinate (x, y, z) resulting in three different models. The input of the model was a 30-days time serie of the SGDP4 distance error. The data they used comes from three objects in low

earth orbit (LEO): a research CubeSat (QuakeSat by Stanford University, NORAD ID: 27845), a satellite payload (COSMOS 2098, NORAD ID: 20774), and a satellite debris (PEGASUS DEB debris, NORAD ID: 23975). These data was collected from Space-Track API. With this approach, it was obtained an improvement on 30-days orbit prediction of 20-30% in average.

Zhai et al. [28] proposed a combination of PCA and XGBoost model to improve the orbit prediction accuracy. The inputs of the model are prediction duration, predicted movement, position at the initial epoch, velocity at the epoch, drag coefficient, predicted position and predicted velocity. The data used for training and evaluation was sourced from satellite simulation environments. First, they trained an XGBoost model to choose the most appropriate combination of features. Then, based on that parameters, the PCA–XGBoost model was trained,in which PCA was used to reduce the dimensionality of the data. The target variable was the true error prediction, which consist of six elements, three axis for position and other three for velocity, so six models are trained. They concluded that this proposal improves satellite predicition in their simulation enviroment and claimed that its capability of generalitation is good for all six components.

3.4. Identified gaps

After an exhaustive review of the state of the art in the field of machine learning applied to satellite propagation, some unexplored areas have been identified. These gaps represent significant opportunities for development and innovation. Not only do they show the need for more detailed research, but they also offer a chance to significantly contribute to current knowledge. These potencial areas of improvement are the following:

- There are no studies that analyze how to adjust the initial parameters of the SGDP4 model to obtain more accurate predictions of the trajectory of space objects. This lack of research leaves a gap in the knowledge about best practices for configuring this model at the start of its calculations. Optimizing these parameters could have a significant impact on the accuracy of the predictions, but this aspect has not yet been addressed in the scientific literature.
- Current state-of-the-art approaches focus only on data related to the specific satellite, such as orbital elements, position, velocity, and drag coefficient. They overlook important information about celestial bodies in the solar system. This oversight is significant because a satellite's trajectory is influenced by all the forces acting on it, including those from other celestial bodies.
- There are no studies in which models have been trained to predict a specific coordinate instead of all three coordinates of a position. Focusing the training of models on predicting just one coordinate could simplify certain aspects of the problem and potentially improve accuracy in specific cases.
- The difference between training models for a specific time frame versus training them in general is often overlooked. The relevance of this distinction lies in the potential benefit that focused training on predictions at specific time intervals

may offer. If there are unique patterns or characteristics in that time frame, training the model with data from that period helps it learn and predict better based on those specifics. This can improve model accuracy for predictions within that time window.

- The evaluation of the models across different time windows (short, medium and long term) is not made clear.

3.5. Conclusions

In this chapter, a comprehensive review of the state of the art has been conducted, identifying several gaps and areas for improvement. This project will focus on addressing these gaps, aiming to contribute to the advancement in this field as much as possible.

4. Description of the proposal

4.1. Introduction

Considering the deficiencies pointed out in Section 3.4, this study focuses on investigating those gaps to uncover the potential of neural networks and the various factors that can influence them. In this way various architectures have been implemented.

4.2. Neural networks

To achieve the objectives of this project, we have developed several neural network architectures. Although countless models can be defined, our goal is to test a selected number of models that represent significantly different approaches and levels of complexity. We have chosen to evaluate four neural network models. Additionally, we have prepared a version of each of these models with an output size of only one, focusing on a single coordinate as target variable. These models are as follows:

4.2.1 DCN

A densely connected network (DCN), also known as a fully connected network, primarily uses dense layers. These layers connect every input feature to every output feature with a unique weight for each connection. This structure allows the network to learn intricate transformations and complex combinations of data. They are particularly useful for tasks where the features do not have an inherent order or spatial relationships, such as tabular data. However, the large number of connections results in a high number of parameters, which can make training these networks challenging.

Figure 4.1 depicts the final architecture of our dense network. The input size of the first layer is defined by the number of features in the dataset when the network is created. Our implementation has an input layer whose output is then fed to further dense layers. Considering the hidden layers, we have three of them, with decreasing number of units (256, 128 and 64). The final dense layer has an output size equal to 3, corresponding to the predicted features of the future state vector. The dense layers with the exception of the last one (since it produces the final prediction) are followed by the application of the RELU function to enable non-linearity.

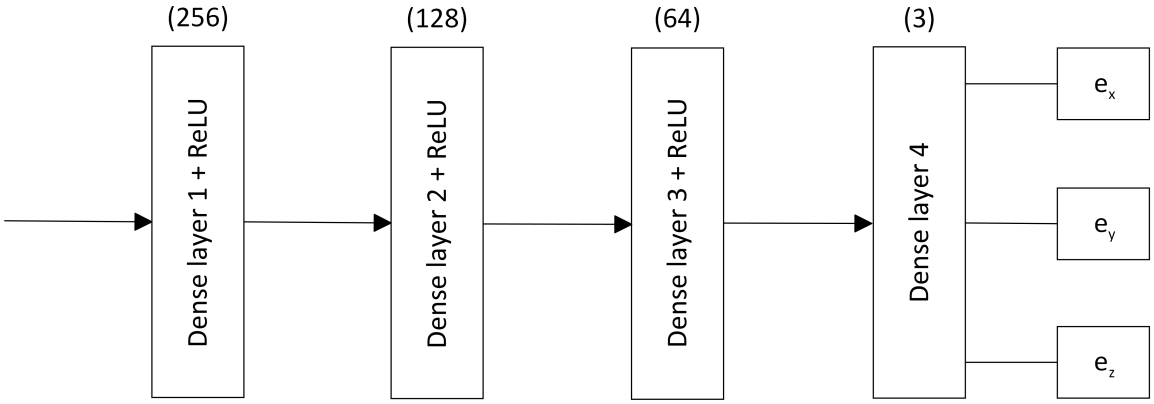

Figure 4.1: Proposed Dense Network

4.2.2 GRU

A GRU (Gated Recurrent Unit) network uses as its main component one or several GRU layers, which are designed to consume sequences of data and produce an output for each element in the sequence. Each element from the sequence is combined with an internal state of the layer (which is also updated with every application) and the output of the former element, all through operations that involve trainable parameters. Therefore, the output corresponding to the last element of the sequence, as well as the inner state after the last application, will be influenced by the entire sequence, making GRU layers ideal to process sequential data. Since a GRU layer produces an output for each input element, an input of shape (n, m) will result in an output of shape (n, x) where x is the configurable size of the output features, enabling the concatenation of several layers. When the last layer is applied, in order to obtain a single vector that represents the entire sequence, it is typical to retain the output or inner state corresponding to the last element of the sequence and ignore the rest [7].

GRU networks have been applied to all kinds of problems in which the input contains some kind of sequence, such as electrocardiogram classification [19] or short-term power load forecasting [29].

One can see our GRU network architecture in Figure 4.2. It starts by defining two separate inputs to handle different types of data. The first input handles sequential data related to the positions of celestial bodies and has a shape of (10, n), where n is the number of features in the sequential data. The second input, handles non-sequential data and its shape is defined by the number of non-sequential features in the dataset.

The sequential data is passed through a Gated Recurrent Unit (GRU) layer with 64 units, while the non-sequential data is passed through a dense layer with 64 units.

The outputs from the GRU layer and the dense layer are then concatenated into a single vector. This concatenated vector is subsequently fed into a series of dense layers with decreasing number of units: 256, 128, and 64 units respectively. Each of these layers applies the ReLU activation function, introducing non-linearity to the model. The final dense layer has an output size of 3, corresponding to the predicted error vector for the future state.

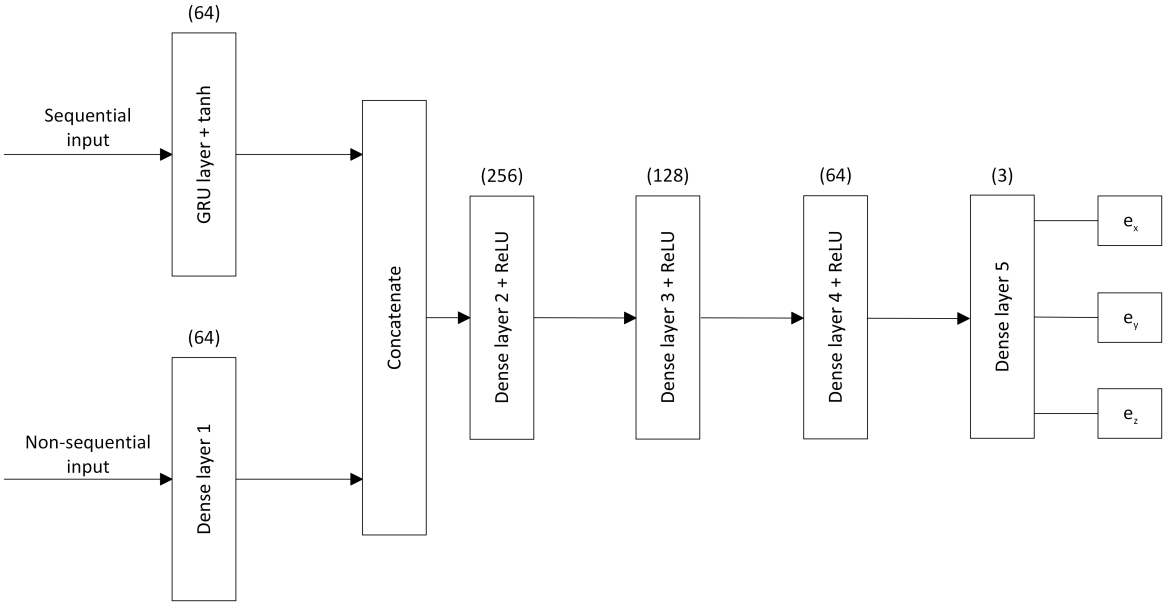

Figure 4.2: Proposed GRU Network

4.2.3 Residual Dense

Residual Networks (ResNets) are a type of artificial neural network architecture that addresses some of the limitations of traditional dense networks, particularly the issue of vanishing gradients. Developed to improve the training of deeper networks, ResNets introduce shortcut connections, also known as skip connections, which allow the input to bypass one or more layers and be added directly to the output [10, 11].

In a ResNet, the core building block is the residual block. Each residual block consists of a series of convolutional layers, batch normalization, or activation functions, followed by the addition of the block's input to its output. This addition is what constitutes the "residual" connection. By explicitly allowing gradients to flow through these shortcut connections, ResNets mitigate the vanishing gradient problem, making it feasible to train networks with many more layers than previously possible.

ResNets are particularly effective in tasks that benefit from deep learning architectures, such as image classification, object detection, and recognition tasks. They excel at learning complex representations from data while maintaining efficient gradient propagation during training. The primary advantage of ResNets is their ability to achieve high performance on deep learning tasks without suffering from the degradation problem that often plagues very deep networks.

In our implementation, as can be seen in Figure 4.3 the network starts with a densely connected layer that takes the input from the dataset. This first dense layer has 128 units and utilizes the relu activation function.

Following this, we introduce a skip connection, where the output of the first dense layer is added to the output of another dense layer with the same number of units and the same activation function. The network then moves to a second dense layer, which has 256 units and also uses the relu activation function.

Another skip connection follows the second dense layer. Here, the output of the second dense layer is added to the output of an additional dense layer with 256 units and relu activation, similar to the first skip connection. After this, the network includes a third dense layer with 128 units and relu activation, followed by a third skip connection that adds the output of this layer to another dense layer with the same configuration.

Finally, the network has a fourth dense layer with 64 units and relu activation, leading up to the output layer. The output layer has 3 units and does not use an activation function since it produces the final prediction corresponding to the predicted features of the future state vector.

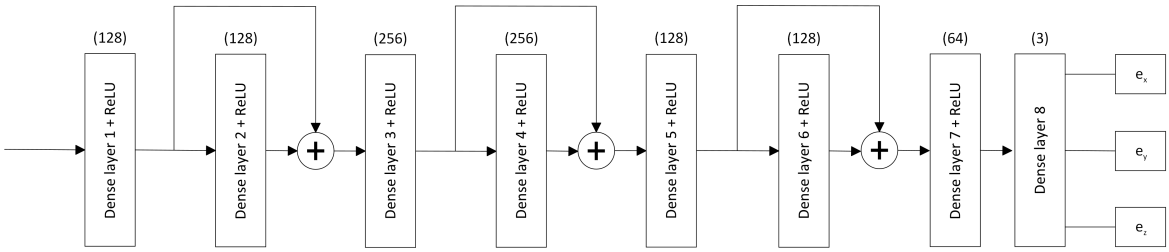

Figure 4.3: Proposed Residual Dense Network

4.2.4 Residual GRU

The philosophy of ResNets was applied to our GRU network, as Figure 4.4 depicts. The input size of the first layer in our network is defined by the number of features in the dataset. In this implementation, we handle two types of inputs: sequential and non-sequential data. For the sequential data, an input layer is defined with a shape corresponding to the sequence length (10) and the number of features. This input is processed by a GRU (Gated Recurrent Unit) layer with 64 units and tanh as activation function.

For the non-sequential data, another input layer is defined with a shape matching the number of non-sequential features. This input is processed by a dense layer with 64 units and no activation function, transforming the non-sequential input data.

The outputs from the GRU layer and the dense layer are concatenated to form a combined input, which is then passed through a series of residual dense layers with decreasing numbers of units. The first dense layer in this sequence has 256 units and uses the ReLU activation function. A residual connection is then introduced by adding the output of another dense layer with 256 units (also followed by ReLU) to the output of the previous layer. This pattern is repeated with a dense layer of 128 units followed by a ReLU function and a residual connection from another 128-unit dense layer. Finally, a dense layer with 64 units and a ReLU activation function processes the combined input further.

The final dense layer has an output size of 3, corresponding to the predicted error in the coordinates. This layer does not use an activation function since it produces the final prediction directly.

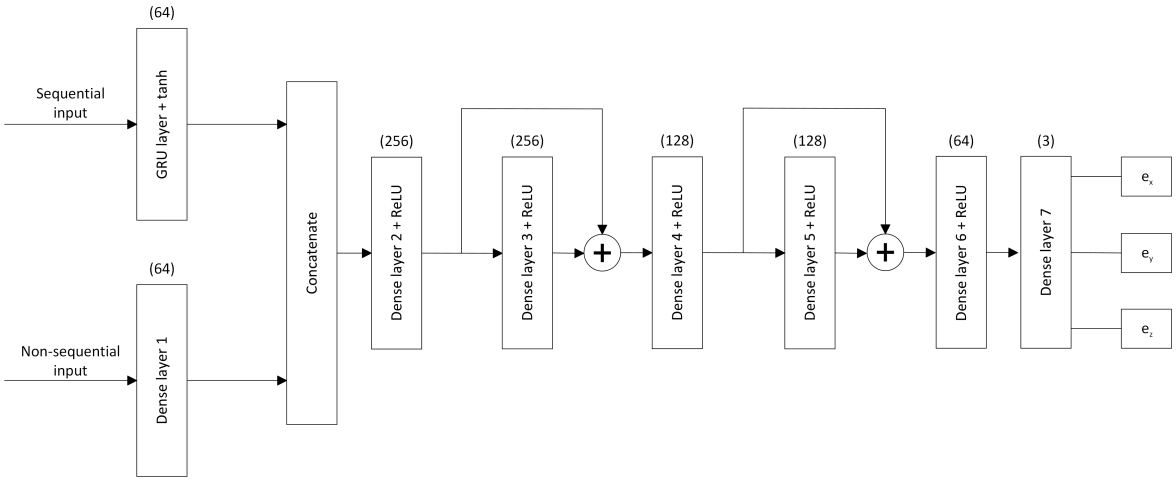

Figure 4.4: Proposed Residual GRU Network

4.3. Conclusions

In this chapter, the neural network architectures developed during the project have been presented and described.

5. Experimentation

5.1. Introduction

In this chapter, the technologies and equipment used in the development of the project are detailed, along with an exhaustive description of the data employed and the experiments conducted. The purpose of this chapter is to provide a clear and comprehensive understanding of the technical and methodological resources implemented, as well as the experimental process that supported the research.

5.2. Technologies and equipment

The project was implemented using the R programming language, which is well-known for its powerful statistical computing and graphics capabilities. We highlight the use of three packages:

- **asteRisk**[1]: This package offers different models for orbit propagation, including the SGDP4, as well as models for predicting the positions of planets and stars.
- **TensorFlow**[2]**/Keras**[3]: These packages were use for building and training deep learning models. TensorFlow, an open-source machine learning library, coupled with Keras, its high-level neural networks API, allowed for the construction of complex neural network architectures and facilitated efficient training and evaluation processes.
- **ggplot2**[4]: This package was used for data visualization. ggplot2, part of the Tidyverse collection, provided a flexible and elegant system for creating complex plots from data in a dataframe. Its capabilities for customizing graphics and layering visual elements made it an essential tool for visually exploring and presenting data insights.

To meet the project's computational demands, we employed a high-performance server with the following specifications: an Intel i9-9900k processor, 64 GB of RAM, an NVIDIA GeForce RTX 3080ti graphics card, and Windows 10 as the operating system.

This combination of advanced software packages and high-performance hardware ensured the project could efficiently manage and process the complex computational tasks required, leading to robust and reliable outcomes.

[1] https://cran.r-project.org/package=asteRisk
[2] https://www.tensorflow.org/
[3] https://keras.io/
[4] https://ggplot2.tidyverse.org/

5.3. Experimental design

The dataset used for this work comes from one originated in the Final Degree Project "Machine Learning aplicado a la propagación de alta precisión de trayectorias de satélites" [6].

These data were provided by Instituto de Estadística y Cartografía de Andalucía (IECA)[5], specifically referencing the Córdoba station. The dataset contains position and velocity data from the Kosmos 2514 satellite, which is part of the GLONASS (Global Navigation Satellite System) constellation. GLONASS provide global satellite navigation services similar to the GPS system. The data covers the period from September 2020 to December 2022 and includes information on predictions at intervals of 30 minutes, 1 hour, 2 hours, 5 hours, 10 hours, 24 hours, 48 hours, 120 hours, 240 hours, and 720 hours.

For simplicity, when a vector magnitude is mentioned, it will refer to all its components. For example, when talking about position, we must think about each of its coordinates, X, Y and Z. Besides, the term *initial date* will be used to indicate the date and time from which we want to make the prediction, that is, the initial moment of the prediction. Here are the columns of the described dataset:

- Initial date
- Position on the initial date
- Velocity on the initial date
- Position of the solar system planets on the initial date
- Lunar libration on the initial date
- Prediction date
- Predicted position with SGDP4
- Predicted movement with SGDP4
- Prediction error with SGDP4 (target variable)

From the columns of dates in this dataset, two additional datasets were created. These new datasets track the positions of specific celestial bodies in the solar system on those dates.

The first dataset focuses on the Sun, the Moon, and Jupiter. These bodies were chosen because their gravitational forces significantly impact Earth and its satellites due to their large masses and proximity.

The second dataset includes the Sun, Venus, the Moon, Mars, and Jupiter. Adding Venus and Mars provides a more comprehensive view of the gravitational influences affecting satellites around Earth.

Both datasets provide sequences of 10 positions for each celestial body. These positions are evenly spaced in time, covering the period from a initial date to a prediction

[5] https://www.juntadeandalucia.es/institutodeestadisticaycartografia/

date.

These data is readily available thanks to precise models like NASA's JPL DE440, which can quickly and accurately calculate the positions of celestial bodies. Including this detailed data allows the network to learn more about the varying influences of different celestial bodies. Factors such as a body's mass and distance from the satellite affect the gravitational force it exerts.

5.4. Informal testing

5.4.1 Orbital parameter optimization

Before training the neuronal network models we tried to develop an optimization method aimed at refining the orbital parameters to enhance the accuracy of the SGDP4 model. The method allowed significant customization in the optimization process through two primary parameters: the number of data points used and the specific weight assigned to each of these points. The weighting mechanism is important because it enables an emphasis on data points that are considered more reliable or significant. For instance, greater weights can be allocated to points at the extremes or the central point of a trajectory arch.

The core of our method involves calculating an error function that takes into account different weights for the errors depending on the point of the trajectory. This approach ensures that the optimization process to be flexible and be able to adapt to prioritize certain aspects of the trajectory. Moreover, our method can also consider the error in the velocity, adding another layer of customizing to the optimization.

In the process, various combinations of weights for the trajectory points were tested, using different number of points (2, 3, 5, and 7). Additionally, the possibility of assigning a weight to the speed in the calculations was considered.

The objective was to determine if these variations could reduce the prediction error. However, after conducting several experiments and analyzing the results, it was found that the prediction error did not decrease significantly, despite the high computational cost involved in the optimization.

5.4.2 Hyperparameter selection

During the experimentation process, informal tests were conducted to select various key hyperparameters. These hyperparameters included the number of training epochs and the batch size.

The informal tests involved adjusting and evaluating different configurations of these parameters to determine which provided the best model performance. For instance, different numbers of epochs were tested to find a balance between training time and model accuracy. Similarly, various batch sizes were evaluated to optimize computational efficiency and training convergence.

These tests helped identify the most suitable configurations, thereby effectively improving models' performance.

5.5. Experimental results

The developed models are designed to predict the error committed by the SGDP4 propagator. To obtain a more accurate prediction of the satellite's position, it is necessary to correct this error. This is achieved by adding the predicted error to the result generated by the SGDP4 propagator.

The whole process can be summarized in the following steps:

1. **Initial propagation**: Use the SGDP4 propagator to obtain a preliminary prediction of the satellite's position.
2. **Error prediction**: Apply one of the developed models to calculate the error associated with that initial SGDP4 prediction.
3. **Prediction correction**: Add the predicted error to the position estimated by SGDP4. This correction adjusts the initial prediction, bringing it closer to the satellite's actual position.

The corrected satellite position, which incorporates both the initial SGDP4 estimate and the error-based correction, provides a more accurate estimation of the satellite's location. This process is detailed in the following picture.

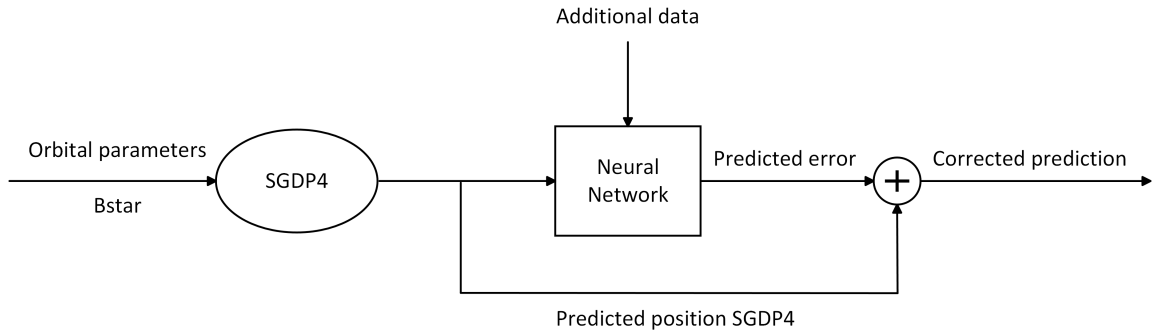

Figure 5.1: Correction of SGPD4 prediction

Before moving on to the models, it's worth mentioning data preprocessing. In this case, robust scaling was used, which is a scaling technique that adjusts the features of the data to have a more uniform distribution. This method is especially useful when the data contains outliers, as it uses the median and the interquartile range instead of the mean and standard deviation, as other scaling methods do. Robust Scaler transforms features by subtracting the median and dividing by the interquartile range (IQR). Given a dataset $X = \{x_1, x_2, \ldots, x_n\}$, the robust scaling is defined by the following formula:

$$X_{\text{scaled}} = \frac{X - \text{median}(X)}{\text{IQR}(X)} \tag{5.1}$$

where:

median(X) is the median of the dataset X.

IQR(X) is the interquartile range of X, calculated as the difference between the third quartile (Q_3) and the first quartile (Q_1):

$$\text{IQR}(X) = Q_3(X) - Q_1(X) \tag{5.2}$$

The following image[6] illustrates the impact of Robust Scaling on different data distributions.

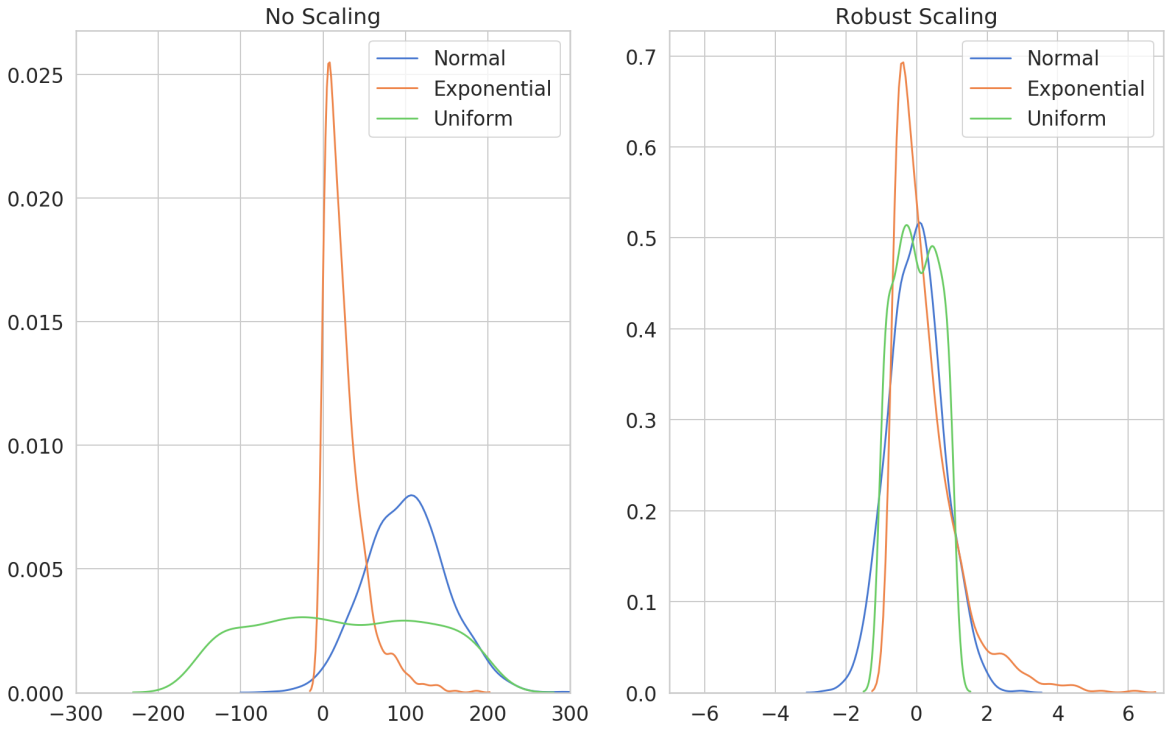

Figure 5.2: Robust Scaling

Now, the trained models will be discussed, explaining which parameters they receive and evaluating the validity and effectiveness of their use.

When it is mentioned that a model receives a position, a displacement, or a velocity, it means that it receives the 3 coordinates or components of that magnitude (X, Y, and Z) as input. We proceeded similarly for other magnitudes, such as lunar libration. Dates will be considered in UNIX format.

All models receive as input parameters the initial date of the prediction, the position of the satellite on that date, and the date on which to make the prediction. Thus, we refer to them as base parameters.

For training all the models in the experiment, the hold-out technique was used for data splitting. An equitable division was chosen, with 50% of the data assigned to the training set and the remaining 50% to the test set. The Adam optimizer was selected

[6]Source: https://curiousily.com/posts/hackers-guide-to-data-preparation-for-machine-learning/

to dynamically adjust the learning rates, accompanied by the MSE (Mean Squared Error) loss function. Additionally, a callback was implemented to automatically save the best model based on the MAE (Mean Absolute Error) metric, thus ensuring the selection of the most accurate model during the training process.

To obtain a reliable indicator of model performance, 10 training sessions were conducted, each using a different seed, ranging from seed 667 to 676. After these trainings, the mean error was calculated, considering the SGDP4 correction in all these 10 models. These trainings and calculations aim to accurately assess the variability and consistency of the models' performance, ensuring that the obtained results are not the product of a single specific run, but rather reflect a general and robust performance.

The experimentation began with an experiment on the DCN (Densely Connected Network) models, evaluating various factors that could influence the model's performance. Specifically, two main aspects were investigated:

- **Inclusion of the satellite's initial velocity**: The impact of incorporating the satellite's initial velocity as an input variable in the model was analyzed. Results obtained by including this information were compared with those where it was omitted, to determine if the initial velocity provided a significant improvement in prediction accuracy.
- **Type of model input, predicted position or displacement**: It was examined whether it was more effective for the model to receive as input the predicted position of the satellite at the prediction instant or the predicted displacement during that time interval. This analysis aimed to identify which of these two forms of input provided better performance in the model's ability to make accurate predictions about the satellite's trajectory.

The training of these models was conducted over 100 epochs with a batch size of 128.

After conducting the initial experiments, it was decided to introduce the positions of the celestial bodies in the solar system in the initial date as input parameters. This was done to study the impact of including different celestial bodies on the accuracy of the obtained results. The trajectory of a satellite is determined by all forces acting upon it, most notably the gravitational influences exerted by these celestial bodies. The gravitational fields of planets, moons, and even smaller bodies within the solar system can significantly affect a satellite's path. By incorporating the positions of these celestial bodies into trajectory models, we allow the network to learn these gravitational influences, thereby enhancing the precision of the trajectory predictions.

The gravitational effects of celestial bodies on a satellite's trajectory are not uniform; they vary based on several factors. Primarily, the mass of the celestial body and its distance from the satellite play pivotal roles. For instance, the gravitational pull of a massive planet like Jupiter can cause noticeable deviations in a satellite's course. Conversely, smaller bodies, such as Europa (a moon of Jupiter), or those located farther away have a less pronounced effect. Therefore, the inclusion of data from these bodies enables the model to adjust predictions based on the varying strengths of gravitational forces encountered along the satellite's journey.

In this way, it is essential to analyze which combination of celestial bodies' data is most beneficial for the predictive model. Not all celestial bodies exert a significant influence on every satellite, and the relevance of a particular body's data may depend on the satellite's specific mission profile and orbital parameters.

These experiments were carried out with three specific variants:

All the planets in the solar system, the Moon, and lunar libration: In this variant, the positions of all the planets in the solar system were considered, as well as the position of the Moon and the effects of its libration. This approach aimed to include as many celestial bodies as possible to observe if the inclusion of more data would result in a significant improvement in accuracy.

The Sun, Venus, the Moon, Mars, and Jupiter: In this second variant, some of the most influential planets were specifically selected along with the Moon and the Sun. This selection was based on their sizes and relative proximity to Earth, with the hypothesis that these bodies have a notable impact on the calculations due to their gravitational forces.

The Sun, the Moon, and Jupiter: The third variant further reduced the number of celestial bodies considered, focusing on the three most influential due to their mass or distance.

As in the previous case, the impact of including the initial velocity, as well as the effect of considering the final position of satellites compared to the displacement experienced, was studied.

The resulting models from these variants were trained in the same way as the previous ones, with 100 epochs and a batch size of 128.

In previous models, we only considered the positions of celestial bodies at the start of the satellite's trajectory. However, the gravitational effects of these bodies impact the satellite throughout its entire path. To improve our model, we should include a sequence of the positions of these celestial bodies from the initial point to the prediction point.

By providing this sequence, the model may better capture the continuous effects of gravitational forces. These forces change as both the satellite and the celestial bodies move, so understanding their ongoing interaction can enhance prediction accuracy.

At this point, GRU network models were introduced. Sequences of positions from the second and third variants were used, which include the positions of the Sun, Venus, the Moon, Mars, and Jupiter, or the positions of the Sun, the Moon, and Jupiter. Sequences with all celestial bodies were not considered because previous experiments determined that the incorporation of data from all planets did not improve the performance of the model. Additionally, the training cost would be very high if all celestial bodies were included.

The training parameters configuration remained constant, using 100 epochs and a batch size of 128.

Although there were previous models that improved the predictions made with the SGDP4 model, the improvement was not significant enough to justify the training

cost for such an enhancement. Thus, charts with the errors are not included.

As the results obtained were not enough good, there was still room for improvement. Therefore, it was decided to employ residual networks (ResNets). To start, Residual Dense was used, conducting a study similar to the one performed with conventional Dense networks but increasing the number of iterations to 500 and maintaining the batch size at 128.

Below, the charts with the results will be displayed. Please note that the vertical axis of the charts is in logarithmic scale. To understand the legends of the charts, it is important to clarify the meaning of the labels:

- **mov**: receives the movement.
- **pos**: gets the predicted position.
- **planets**: takes the position of all celestial bodies.
- **lib**: takes lunar libration.
- **smj**: receive the position of the Sun, Moon, and Jupiter.
- **svmmj**: gets the position of the Sun, Venus, Moon, Mars, and Jupiter.
- **gru**: indicates that the model received sequential data.
- **XYZ**: indicates that the information is separated by coordinates.
- **GPT**: indicates that the model is trained for specific times.

First, the experiment was carried out without considering celestial data:

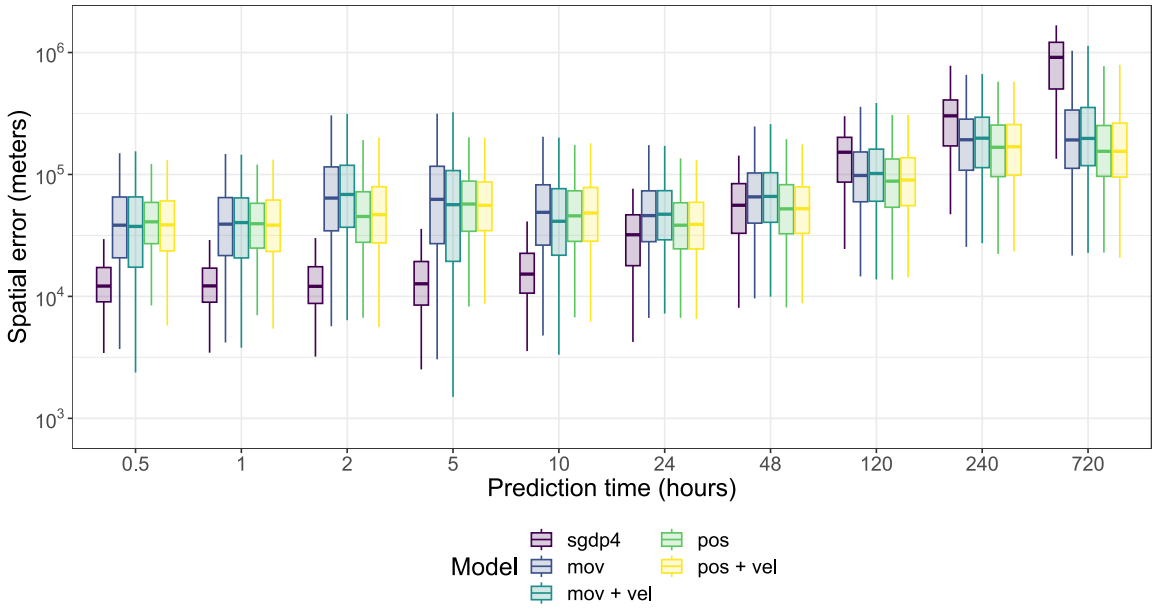

Figure 5.3: Spatial error without celestial data

As can be seen in the image, it is from the 48-hour predictions onwards that some models start to outperform SGDP4, with the improvement being very clear in

the 720-hour (30-day) predictions. This represents a significant advance for long-term predictions.

Then, data from all planets in the solar sistem was added.

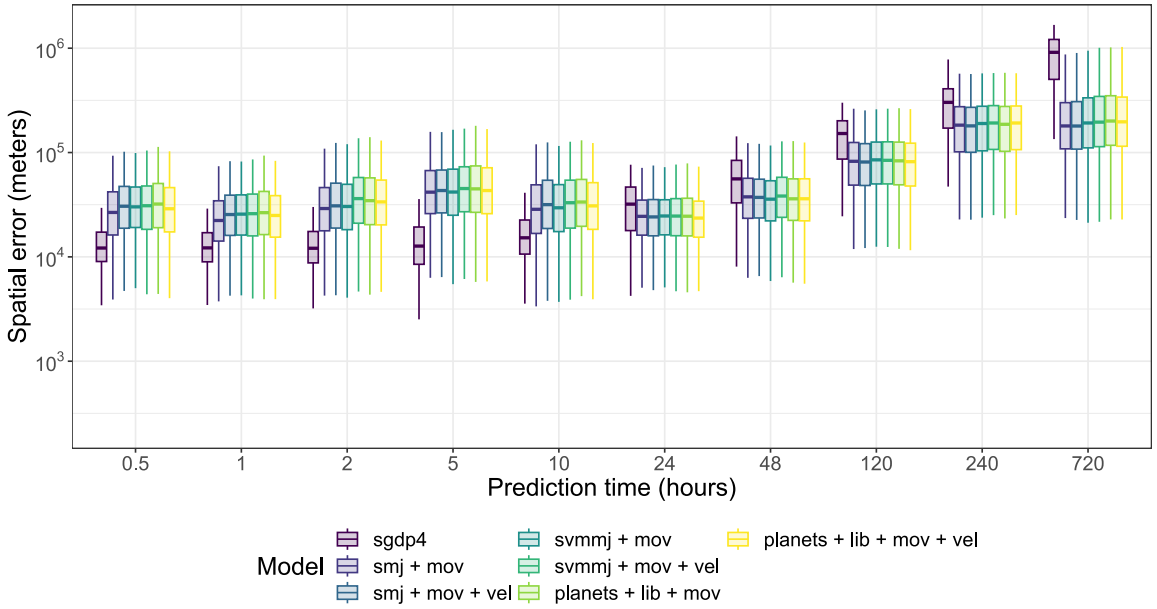

Figure 5.4: Spatial error with celestial data

Now, we can observed some improvement from 24 hours. However, we still need to enhance our short-term predictions.

Subsequently, Residual GRU was implemented, following the same approach as with the initial GRU networks. In this case, the batch size was kept at 128, but the number of epochs was adjusted to 300 to evaluate the model's performance under these conditions.

The use of ResNets was based on the ability of these networks to handle degradation problems and improve model accuracy through the incorporation of residual connections. These connections allow the gradient to flow more easily through the network, facilitating the training of deeper models.

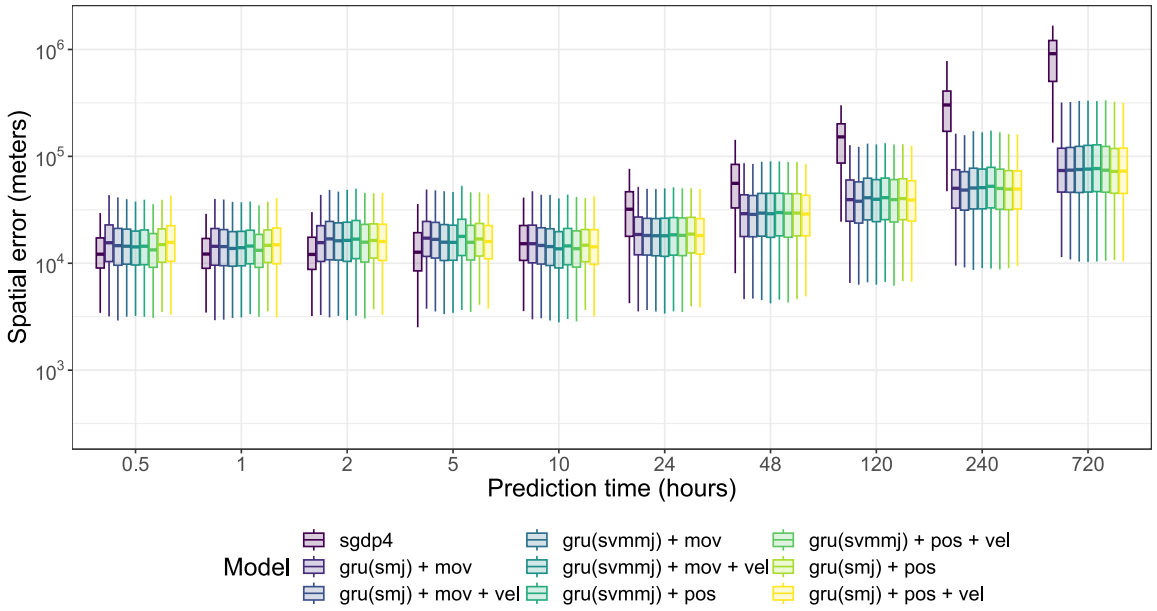

Figure 5.5: Spatial error with sequential celestial data

It is observed that some of the Residual GRU models begin to improve predictions after 10 hours, although not very significantly. However, in long-term predictions, there is a substantial improvement.

The next step was to select the best-performing architecture and apply coordinate separation, resulting in the creation of three models instead of one. This process was carried out for the models that showed the best performance, which were all of the Residual GRU architecture. Thus, the models named Residual GRU XYZ were developed.

Coordinate separation involved dividing the predictions into three distinct components: X, Y, and Z. At first glance, the models may perform similarly. However, since we are dealing with neural networks, the loss from one coordinate might negatively impact the others. This can lead to worse overall performance, as errors in predicting one coordinate can affect the predictions of the others.

Therefore, it's important to see if training a separate model for each coordinate, despite being more costly, improves the prediction results. By having individual models, each one can focus on the specific characteristics and patterns of its coordinate without interference from others. This specialization can lead to more accurate and reliable predictions.

These Residual GRU XYZ models were trained with 300 iterations and a batch size of 128, maintaining consistency with the previously used training parameters. The results are as follow:

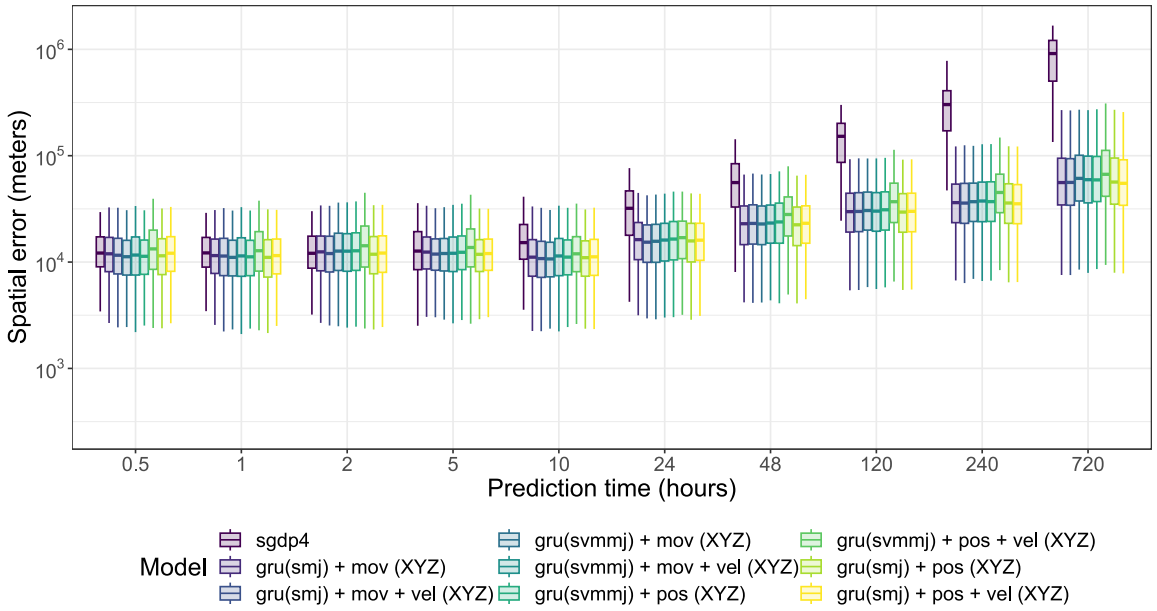

Figure 5.6: Spatial error in separate-coordinate models

The improvement in predictions at the 10-hour mark is clearly visible. Some models even show a slight enhancement in the 5-hour predictions. Moreover, in long-term predictions, there is an improvement compared to models without coordinate separation. This indicates that separating coordinates has a positive impact on both short-term and long-term prediction accuracy.

Furthermore, long-term predictions using the SGDP4 model often become significantly distorted, indicating potential for improvement in extended period predictions. Short-term predictions present their own challenges, being less prone to severe distortions but difficult to improve in accuracy.

To enhance the SGDP4 model, we could explore training models for different time windows. Specialized models for short-term, medium-term, and long-term predictions allow for fine-tuning parameters and choosing algorithms that best fit each period. Studying these specialized models can help determine if the additional effort and resources lead to better performance compared to a general approach.

The architecture showing the best accuracy was the Residual GRU XYZ with data from the Sun, Venus, the Moon, Mars, and Jupiter. Based on this architecture, 10 models were trained, each specializing in a specific time band: 30 minutes, 1 hour, 2 hours, 5 hours, 10 hours, 24 hours, 48 hours, 120 hours, 240 hours, and 720 hours. In this approach, for a specific prediction time of SGDP4, the corresponding specialized model is used, such as applying the 10-hour band model for an 8-hour prediction.

Each of the 10 models was trained with 300 iterations and a batch size of 128. Here we have the results:

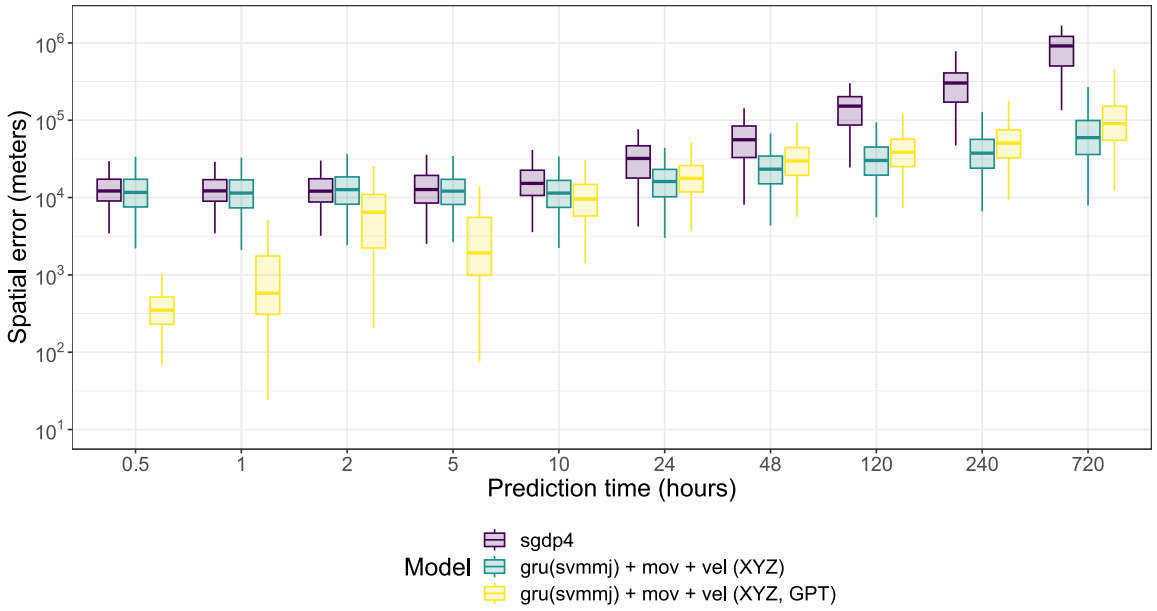

Figure 5.7: Spatial error in time-band-focused model

The new model has generally shown improved predictions compared to previous ones. This improvement is especially noticeable in the short term, where it has achieved better results across all time bands. This is a significant achievement, as previous models have not managed to perform well in all short-term time bands simultaneously.

In the medium and long term, the new model also outperforms the SGDP4. However, it is important to note that it does not surpass its equivalent version that was trained without using time bands. Despite this, the overall enhancement in prediction accuracy marks a substantial advancement in the model's capabilities.

It can be concluded that by incorporating sequential information from some planets, a considerable reduction in the error of the SGDP4 model is achieved. This finding opens new opportunities for future research that aims to apply and generalize the model to a larger number of satellites. The improvement in the accuracy of the SGDP4 model when using sequential data highlights the importance of considering this approach in subsequent studies, allowing for significant advancements in the prediction and analysis of satellite behavior in various contexts.

5.6. Conclusions

In this chapter, the experimentation carried out has been presented, analyzing the results obtained in detail and highlighting those of greatest relevance.

6. Conclusions and future work

This research has focused on the application of neural networks to improve the accuracy of the SGDP4 orbital propagation model. Simplified Perturbations Models, like the SGDP4, are extremely fast and have a very low computational cost; however, their significant loss of accuracy in long-term predictions makes them unsuitable for tasks requiring high precision.

Initially, tests were developed to optimize the orbital input parameters of the SGDP4 model in order to increase its accuracy. Despite the efforts, the results obtained were not very promising, leading to the abandonment of this line of exploration.

The value and novelty of our work lie in the integration of the SGDP4 propagator with neural network models. For the research, we have tested various combinations of positions of celestial bodies within the solar system, developed specific models for each coordinate, and created models for predictions in specific time intervals. These three aspects represent innovative approaches that have not been previously explored. We conclude that the application of neural networks to improve the performance of the SGDP4 propagator has significant potential, as demonstrated in various configurations where notable error reductions are achieved.

Here are several ideas proposed for future work:

1. **Study of the computational cost of orbital parameter optimization**: Conduct an exhaustive analysis of the computational cost involved in optimizing orbital parameters.
2. **Validation of models with other satellites**: Validate the use of these trained models to predict the trajectories of other satellites, especially those in nearby orbits. Additionally, explore the feasibility of developing a general-purpose prediction model to assess its applicability and robustness in various contexts.
3. **Inclusion of other astronomical data**: Study the effect of including data from astronomical phenomena, such as solar storms, that may affect the satellite's movement.

I would like to express my satisfaction with the results obtained from this research. The findings are highly relevant and promise to make a significant contribution to the field of study. Given their importance, we are preparing a manuscript for publication in a journal classified in the Q2 quartile. We believe that these results will not only enrich the existing literature but also open new lines of investigation for future studies.

7. Bibliography

[1] Daniel Ayala, Inma Hernández, David Ruiz, and Miguel Toro. Tapon: A two-phase machine learning approach for semantic labelling. *Knowledge-Based Systems*, 163:931–943, 2019. ISSN 0950-7051. doi: https://doi.org/10.1016/j.knosys.2018.10.017. URL https://www.sciencedirect.com/science/article/pii/S0950705118305033.

[2] Daniel Ayala, Agustín Borrego, Inma Hernández, and David Ruiz. A neural network for semantic labelling of structured information. *Expert Systems with Applications*, 143:113053, 2020. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2019.113053. URL https://www.sciencedirect.com/science/article/pii/S0957417419307705.

[3] Rafael Ayala, Daniel Ayala, Lara Sellés Vidal, and David Ruiz. asterisk - integration and analysis of satellite positional data in r. *The R Journal*, 15:34–54, 2023. ISSN 2073-4859. doi: 10.32614/RJ-2023-023. URL https://doi.org/10.32614/RJ-2023-023.

[4] Roger R. Bate, Donald D. Mueller, and Jerry E. White. *Fundamentals of astrodynamics*. Dover Publications, 1 edition, 1971. URL https://books.google.es/books?id=UtJK8cetqGkC.

[5] Dirk Brouwer. Solution of the problem of artificial satellite theory without drag. *Astronomical Journal*, 64:378, nov 1959. doi: 10.1086/107958. URL https://ui.adsabs.harvard.edu/abs/1959AJ.....64..378B.

[6] José Calderón Valdivia. Machine Learning aplicado a la propagación de alta precisión de trayectorias de satélites (Trabajo de Fin de Grado Inédito). *Universidad de Sevilla*, 2023. URL https://hdl.handle.net/11441/149000.

[7] Kyunghyun Cho, Bart Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. 06 2014. doi: 10.3115/v1/D14-1179.

[8] Roberto Flores, Burhani Makame Burhani, and Elena Fantino. A method for accurate and efficient propagation of satellite orbits: A case study for a molniya orbit. *Alexandria Engineering Journal*, 60(2):2661–2676, 2021. ISSN 1110-0168. doi: https://doi.org/10.1016/j.aej.2020.12.056. URL https://www.sciencedirect.com/science/article/pii/S1110016821000016.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling, editors, *Computer Vision – ECCV 2016*, pages 630–645, Cham, 2016. Springer International Publishing. ISBN 978-3-319-46493-0. URL https://link.springer.com/chapter/10.1007/978-3-319-46493-0_38.

[12] Felix R. Hoods and Ronald L. Roehrich. *SPACETRACK REPORT NO. 3. Models for Propagation of NORAD Element Sets*. U.S. Department of Defense, 1988. URL https://celestrak.org/NORAD/documentation/spacetrk.pdf.

[13] Richard S Hujsak. A restricted four body solution for resonating satellites without drag. *NASA STI/Recon Technical Report N*, 80:24340, 1979. doi: 10.21236/ADA081263. URL https://apps.dtic.mil/sti/citations/tr/ADA081263.

[14] P Golda Jeyasheeli, C Kamaleshwar, and K Sakthi Aswin. Deep learning methods for suicide prediction using audio classification. *Journal of Positive School Psychology*, pages 10479–10485, 2022.

[15] Yoshihide Kozai. The motion of a close earth satellite. *Astronomical Journal*, 64:367, nov 1959. doi: 10.1086/107957. URL https://ui.adsabs.harvard.edu/abs/1959AJ.....64..367K.

[16] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521 (7553):436–444, May 2015. doi: 10.1038/nature14539. URL https://doi.org/10.1038/nature14539.

[17] Creon Levit and William Marshall. Improved orbit predictions using two-line elements. *Advances in Space Research*, 47(7):1107–1115, 2011. ISSN 0273-1177. doi: https://doi.org/10.1016/j.asr.2010.10.017. URL https://www.sciencedirect.com/science/article/pii/S0273117710006964.

[18] Zac Yung-Chun Liu, Scott Tarlow, Mohammad Akbar, Quentin Donnellan, and Darek Senkow. Improved orbital propagator integrated with sgp4 and machine learning. In *35th Annual Small Satellite Conference*, 2021. URL https://digitalcommons.usu.edu/cgi/viewcontent.cgi?article=5067&context=smallsat.

[19] Htet Myet Lynn, Sung Bum Pan, and Pankoo Kim. A deep bidirectional gru network model for biometric electrocardiogram classification based on recurrent neural networks. *IEEE Access*, 7:145395–145405, 2019. doi: 10.1109/ACCESS.2019.2939947.

[20] O Montenbruck, E Gill, and FH Lutze. Satellite Orbits: Models, Methods, and Applications. *Applied Mechanics Reviews*, 55(2):B27–B28, 04 2002. ISSN 0003-6900. doi: 10.1115/1.1451162. URL https://doi.org/10.1115/1.1451162.

[21] Hao Peng and Xiaoli Bai. Limits of Machine Learning Approach on Improving Orbit Prediction Accuracy using Support Vector Machine. In S. Ryan, editor, *Advanced Maui Optical and Space Surveillance (AMOS) Technologies Conference*, page 15, January 2017. URL https://ui.adsabs.harvard.edu/abs/2017amos.confE..15P.

[22] Hao Peng and Xiaoli Bai. Artificial neural network–based machine learning approach to improve orbit prediction accuracy. *Journal of Spacecraft and Rockets*, 55:1–13, 06 2018. doi: 10.2514/1.A34171. URL https://doi.org/10.2514/1.A34171.

[23] Ahmed Refaat, Ahmed Badawy, Mahmoud Ashry, and Omar Adel. High accuracy spacecraft orbit propagator validation. *The International Conference on Applied Mechanics and Mechanical Engineering*, 18:1–9, 04 2018. doi: 10.21608/amme.2018.34732.

[24] Haoli Ren, Xiaolin Chen, Bei Guan, Yongji Wang, Tiantian Liu, and Kongyang Peng. Research on satellite orbit prediction based on neural network algorithm. pages 267–273, 06 2019. ISBN 978-1-4503-7185-8. doi: 10.1145/3341069.3342995. URL https://doi.org/10.1145/3341069.3342995.

[25] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2014.09.003. URL https://www.sciencedirect.com/science/article/pii/S0893608014002135.

[26] Ho-Nien Shou. Orbit propagation and determination of low earth orbit satellites. *International Journal of Antennas and Propagation*, 2014(1):903026, 2014. doi: https://doi.org/10.1155/2014/903026. URL https://onlinelibrary.wiley.com/doi/abs/10.1155/2014/903026.

[27] David A Vallado. *Fundamentals of astrodynamics and applications*, volume 12. Springer Science & Business Media, 2001.

[28] Min Zhai, Zongbo Huyan, Yuanyuan Hu, Yu Jiang, and Hengnian Li. Improvement of orbit prediction accuracy using extreme gradient boosting and principal component analysis. *Open Astronomy*, 31(1):229–243, 2022. doi: doi: 10.1515/astro-2022-0030. URL https://doi.org/10.1515/astro-2022-0030.

[29] Jiaxiang Zheng, Xingying Chen, Kun Yu, Lei Gan, Yifan Wang, and Ke Wang. Short-term power load forecasting of residential community based on gru neural network. In *2018 International Conference on Power System Technology (POWERCON)*, pages 4862–4868, 2018. doi: 10.1109/POWERCON.2018.8601718.